

1) Image Formation

An image is a bidimensional representation of the 3D world, there exist 3 types of representations:

- **Grayscale:** $f(x, y) \in [0, 1]$ continuous function that represents the intensity of the gray level
- **Color Image:** $f_c(x, y, \lambda)$ where λ is the value for each wavelength. A good approximation is to use 3 continuous functions for each pixel color: $f_{R/G/B}(x, y)$ (details later)
- **Digital Images:** as before but $x, y \in \mathbb{N}$. So the values of f are discrete and finite. In this course we use discrete images and 8-bit intensity values.

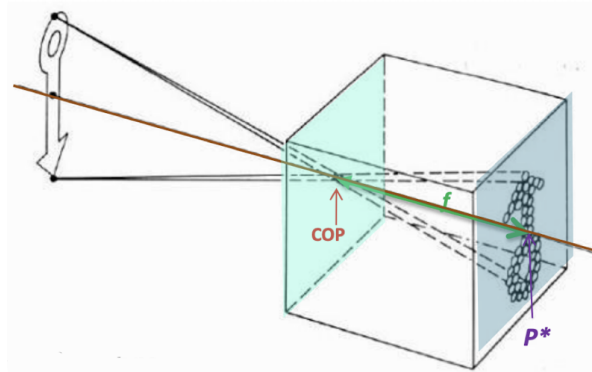
1.1) Intro

Pinhole Model

Put barrier with small hole between object and sensor. The image is mirrored.

Here is some terminology:

- Center Of Projection (COP, C):** pinhole location
- Imaging Plane (R):** plane of sensor
- Focal Distance (f):** distance between COP and R
- Optical Axis:** line passing through COP and perpendicular to R
- Principal Point (P*):** intersection of optical axis and R
- Focal Plane (F):** plane parallel to R containing COP



Pinhole Camera Model

Camera Coordinates

Camera model maps 3D coordinates $\tilde{M} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$ to 2D ones $\tilde{m} = \begin{bmatrix} u \\ v \end{bmatrix}$.

Homogeneous coordinates add an extra dimension in order to normalize the last coordinate

$$M = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \rightarrow m = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

3D World Coordinate System and 2D Camera Coordinate System

Perspective Projection

Since from 3D to 2D some information is lost, the perspective is non-linear. The coordinates get divided by z , so that farther objects appear smaller.

By noticing that $\begin{cases} \frac{f}{Z} = -\frac{u}{X} \\ \frac{f}{Z} = -\frac{v}{Y} \end{cases}$

The relationship between M and m is:

$$m \approx PM$$

$$Zm = Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = Z \begin{bmatrix} -\frac{fX}{Z} \\ -\frac{fY}{Z} \\ 1 \end{bmatrix} = \begin{bmatrix} -fX \\ -fY \\ Z \end{bmatrix} = \underbrace{\begin{bmatrix} -f & 0 & 0 & 0 \\ 0 & -f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_P \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Generalized model:

Dropped Assumptions	Intrinsic Parameters	Extrinsic Parameters
Arbitrary camera position Position of principal point	Focal Length Pixel Size ($p_u, p_v \rightarrow k_u = \frac{1}{p_u}, k_v = \frac{1}{p_v}$) Coordinates of Principal Point (u_o, v_o)	Camera position Viewing Direction

Altogether this gives us 11 Degrees of freedom (5 Intrinsic, 3 rotational, 3 translational)

🕒 **Todo**

Math Model

This model however causes the **loss of information of:**

- Angles
 - Distance and dimension
 - Parallel lines
 - Collinearity of points
- However, straight lines remain straight**

Perspective

The field of view is found as

$$\alpha = 2 \arctan \left(\frac{d}{2f} \right)$$

Close up pictures have $\Delta Z \sim Z$ (grandangolo) and require high fov, while long-distance pictures have $\Delta Z \ll Z$ and require less fov

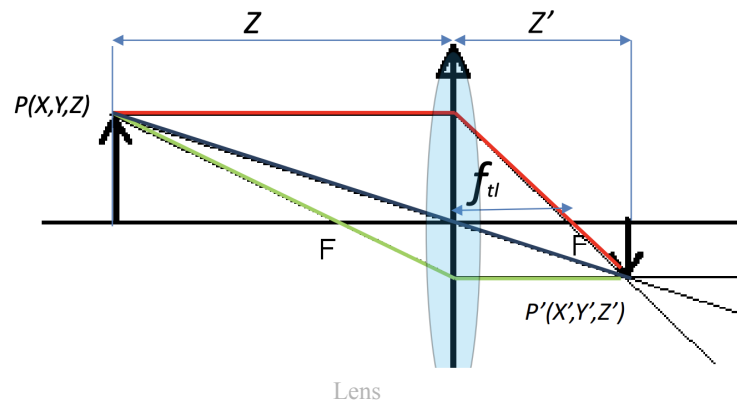
Vertigo Effect (Dolly Zoom)

1.2) Real Cameras

How does the pinhole size (aperture) affect the image?

Pinhole	Small	Large
Amount of light	low	High
Focusing	Good	poor

Lenses:



The lens corrects the path of the rays.

- Any ray parallel to the axis proceeds towards F
- Any ray passing through the center is unchanged
- Any ray passing through F before the lens becomes parallel

If an object at distance Z is at focus ad distance Z' then the following relation (lens equation) holds:

$$\frac{1}{f_{tl}} = \frac{1}{Z} + \frac{1}{Z'}$$

If the distance is different from Z' we enter the **circle of confusion** where the rays don't focus on the plane. If this circle is bigger than the pixel (cmos sensor) then we get blurring.

Lenses gather more light but need to be focused

CMOS Sensors

CMOS light sensitive sensors with a filter in front of them are used to gather the pixel data. A **Color Filter Array (CFA)** is an array of alternating color filters that samples only one color band at each pixel location. The **Bayern pattern** consists of a 2x2 pixel array with 2 green one red and one blue

Camera Properties

1. **Focus (and defocus):** Any object satisfying the lens equation is in focus
2. **Fov and focal length:** smaller fov $\iff$ larger focal length
3. **Exposure:** is the amount of light that enters the sensor, it is controlled by
 1. aperture: pinhole size expressed as the diameter of lens opening $a = f/N$ with N multiples of 2. Aperture also controls DoF: smaller aperture makes bigger circle of confusion, but requires more light (more shutter)
 2. shutter speed: time it stays open. expressed in fraction of seconds. Longer shutter speed, more light but more motion blur

Too much light $\rightarrow$ blurring. Too little $\rightarrow$ diffraction

Theorem (Reciprocity theorem).

The same exposure is obtained with an exposure twice as long and an aperture area half as big

Feature	Aperture ($a=f/N$)	Shutter Speed (Time t)
Definition	The diameter of the lens opening controlled by the diaphragm.	The duration of time the shutter remains open to expose the sensor.
Measurement	Measured in f-numbers (N). Note: A <i>smaller</i> f-number equals a <i>bigger</i> aperture (e.g., $f/2.0$ is a large opening; $f/32$ is small).	Measured in fractions of a second (e.g., $1/60s$, $1/1000s$).
Effect on Light	Large Aperture ($f/2$): Gathers more light. Small Aperture ($f/16$): Gathers less light.	Slow Speed ($1/8s$): More light enters (exposure is proportional to time). Fast Speed ($1/500s$): Less light enters.
Visual Effect	Controls Depth of Field (DoF): • Large Aperture ($f/2$): Shallow DoF. The background is blurred/out of focus. • Small Aperture ($f/16$): Deep DoF. A larger range of distances (foreground to background) is in focus.	Controls Motion Blur: • Slow Speed ($1/8s$): Motion blur occurs (e.g., moving birds or water look soft). • Fast Speed ($1/500s$): Freezes motion (e.g., flapping wings look sharp).
Trade-offs	A small aperture provides sharpness throughout the scene but reduces light, requiring a longer exposure time.	A slow shutter speed gathers more light but introduces the risk of motion blur or camera shake (often requiring a tripod).

Other phenomena:

- Chromatic aberration: different wave lengths get focused at different points, happens mostly in outer lens edges
- Radial Distortion: not ideal lenses deform the grid

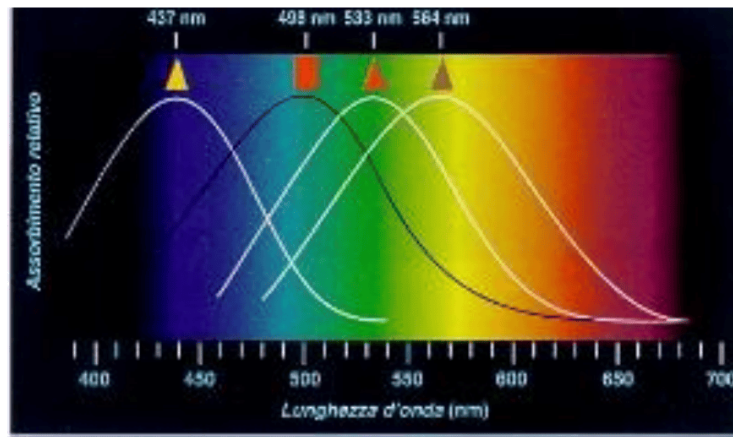
1.3) Color Vision

Our eye acts as a camera: Cornea is lens, crystalline lens controls focus, iris controls aperture, pupil is pinhole

We have 2 photoreceptors:

- Cones: Color vision (band pass filters), 6 million low sensitivity in fovea
- Rods: Grayscale, high sensitivity, 120 million on the sides

Intensity perception is logarithmic, spatial and temporal information has high impact, the color difference has more impact than absolute color



Band Pass Filters

This image shows why green is the most sensible.

Color Spaces

We define the **Gamma (gamut)** of a color space as the set of colors physically realizable in that space.

Additive:

the effect of a color (spectral density $C(\lambda)$) and the cones with response $S_i(\lambda)$ is represented as the integral

$$\hat{a}_i = \int_{350}^{780} C(\lambda) S_i(\lambda) d\lambda.$$

These can be added, to represent a color

This is represented with a 3d vector space (r,g,b independent), however some triplets are negative and therefore cannot be physically represented

CIE RGB 1931:

R,G,B colors represented by dirac delta functions. multiplying these by a value and adding them together represents the colors

Definition 1 (Color Matching Functions (CMF)).

The CMF is a triplet of functions of the wavelength representing the chromatic coordinates $a(\lambda_i)$ of the spectral colors $E_i(\lambda)$ with respect to the primary of the CIE 1931 standard

This means that

$$i = \beta_i = \int_{350}^{780} C(\lambda) \underbrace{\alpha_k(\lambda)}_{i\text{-th CMF}} d\lambda = \sum_j C(\lambda_j) \underbrace{\alpha_k(\lambda_j)}_{i\text{-th CMF}}, \quad i = \{r, g, b\}$$

and finally

$$C(\lambda) = \sum_i \beta_i P_i(\lambda) = \sum_i \left[\sum_j C(\lambda_j) \alpha(\lambda_j) \right] P_i(\lambda)$$

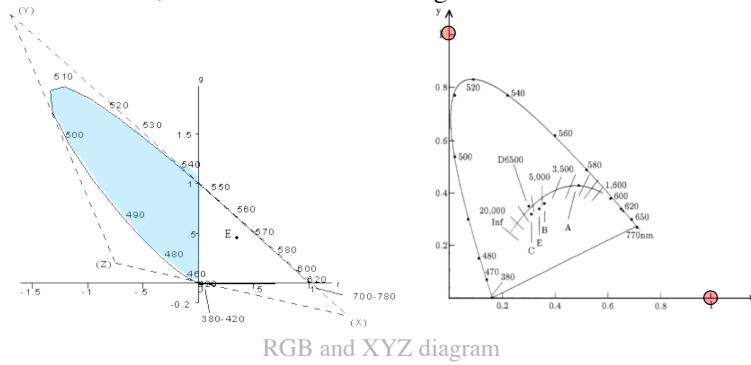
Definition 2 (Chromatic Coordinates).

This is the ratio between the chromatic coordinates and their sum

$$i = \frac{I}{R + G + B}$$

2d space since $r + b + g = 1$ and thus convexity is preserved

The **Horseshoe diagram** shows the intersection of spectral colors curve with the $R + G + B = 1$ plane
Negative colors cannot be realized, colors outside of the diagram cannot be realized



Moreover the McAdam Ellipse represents a part of the color space where our eyes cannot detect the difference.
Uniform color spaces try to make ellipses more uniform

Spazio di Colore	Caratteristiche Principali	Ambito di Utilizzo
RGB CIE 1931	Primary colors represented as Dirac delta functions $P_i(\lambda) = \delta(\lambda - \lambda_i)$ $C(\lambda) = R \cdot P_r + G \cdot P_g + B \cdot P_b$ Where R, B, G are β_i obtained through CMF	Theoretic base for color synthesis
XYZ	Linear TX of RGB	CMF all positive, primaries not physically realizable
CIELUV	Uniform space, the color difference reflects the geometrical distance	monitor
CIELAB	The perceived color differences correspond to the geometric distances	Subtractive synthesis
sRGB		Used by most monitors, Gamut non too large

2) Low level Image Processing Algorithms

2.1) Intro

Sampling and Quantization

Real life objects are captured by the camera and thus sampling arises with sampling period $\Delta x, \Delta y$ that correspond to the pixel sizes, Then these values get quantized

$$f_d(m, n) = Q[f(m, n)] = Q[f(m\Delta x, n\Delta y)]$$

$L = 2^k$ quantization levels correspond to $b = MNk$ bits, rgb images require 3 times as much.

Discrete 2d DFT

the discrete transforms:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$
$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

However in the DFT **the transforms are discrete and periodic**

Theorem 3 (Fourier Transform Separability).

A 2D DFT can be computed by M 1D-DFT (one) for each column and N 1D DFT (one) for each row.

$$F(u, v) = \sum_{x=0}^{M-1} e^{-j2\pi\frac{ux}{M}} \underbrace{\sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\frac{vy}{N}}}_{F' = 1d \text{ DFT}} = \sum_{x=0}^{M-1} F'(x, v) e^{-j2\pi\frac{ux}{M}}$$

Where $F'(x, v)$ is the sum of all 1D DFT done on the Y axis.

Theorem 4 (Separability Theorem).

If the system allows the decomposition $h(x, y) = f(x)g(y)$ then the 2D DFT will be $H(u, v) = F(u)G(v)$

$$h(x, y) = f(x)g(y) \rightarrow H(u, v) = F(u)G(v)$$

Remark.

Do not confuse the separability thm with product $\leftrightarrow$ convolution property! In fact, in general if

$$h(x, y) = f(x, y)g(x, y) \xrightarrow{\mathcal{F}} H(u, v) = F(u, v) * G(u, v)$$

Quick Recap on DFT Properties

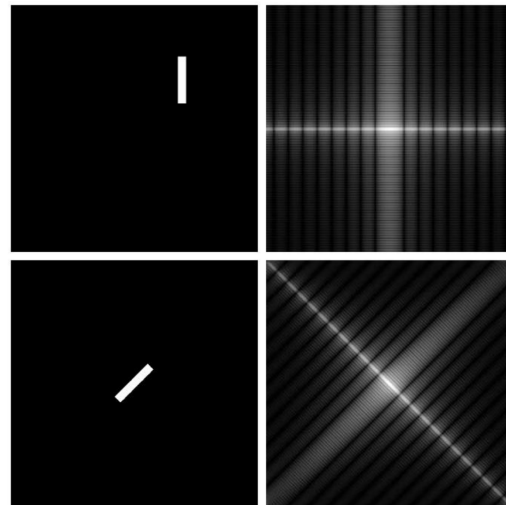
Polar Representation:

$$F(u, v) = |F(u, v)|e^{j\phi(u, v)}$$

Spectrum (Magnitude):

$$|F(u, v)| = \sqrt{\Re[F(u, v)] + \Im[F(u, v)]}$$

Notice how a translation in the image doesn't change the spectrum, however a rotation changes the frequency (it rotates the spectrum)

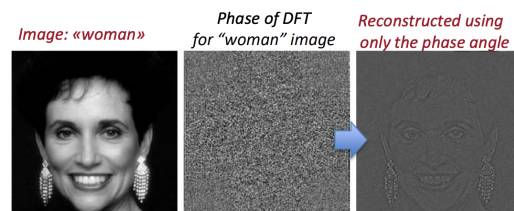


Translation and Rotation

Phase:

$$\phi(u, v) = \arctan \left(\frac{\Re[F(u, v)]}{\Im[F(u, v)]} \right)$$

The phase contains the most info on the image



Example

Convolution/Product:

$$f(x, y) * h(x, y) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) h(x - m, y - n) \xrightarrow{\mathcal{F}} F(u, v) H(u, v)$$

See also this [Theorem 4 \(Separability Theorem\)](#).

Separability:

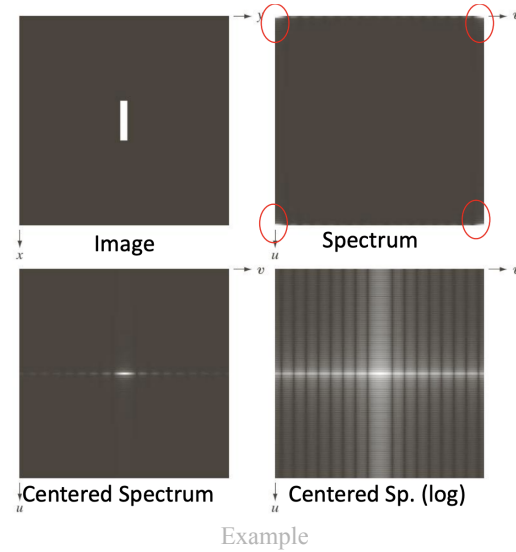
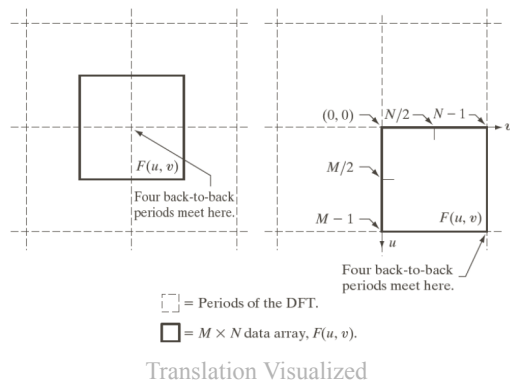
See [Theorem 4 \(Separability Theorem\)](#).

Translation To Center of The Frequency Rectangle:

$$f(x - M/2, y - N/2) \xrightarrow{\mathcal{F}} F(u, v)(-1)^{u+v}$$

In particular the power is concentrated in the lower frequencies (start and end of period), the shift allows to have the energy in the middle region.

REMEMBER TO CENTER THE IMAGE, that is to shift by multiplying by $(-1)^{x+y}$



Gaussian: (dft of gaussian is gaussian)

$$A2\pi\sigma^2 e^{-2\pi^2\sigma^2(t^2+z^2)} \xrightarrow{\mathcal{F}} A e^{-(\mu^2+v^2)/2\sigma^2}$$

Band-limited Images:

Theorem 5 (Band-Limited).

A 2D signal is said to be strictly band limited if its fourier transform is such that

$$F(u, v) = 0 \text{ for } |u| > u_{\max} \\ \text{and} \\ F(u, v) = 0 \text{ for } |v| > v_{\max}$$

2D Sampling Theorem

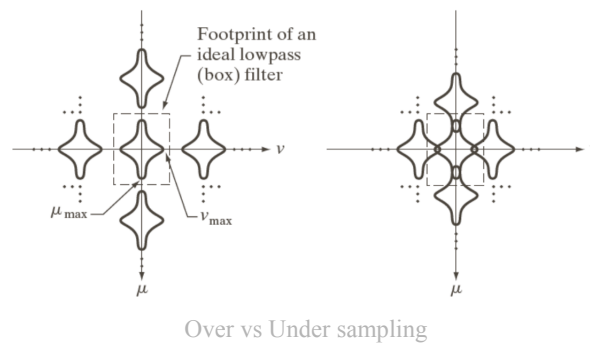
Theorem 6 (2D Sampling Theorem).

Let $f(x, y)$ be a band limited image uniformly sampled over an orthogonal lattice with spacing $\Delta X, \Delta Y$. Then, if the sampling rate is greater than the nyquist rate, $f(x, y)$ can be reconstructed from the sampled values without any loss of information, that is

$$F_x = \frac{1}{\Delta X} > 2u_{\max} \text{ and } F_y = \frac{1}{\Delta Y} > 2v_{\max} \implies F^{-1}(u, v) = f(x, y)$$

Aliasing

Undersampled images will have spectrum overlapping, this is called aliasing



When compressing an image, instead of acting directly with pixel deletion, a LPF will remove aliasing in the final image

Resampling

What we can do:

- Interpolate to zoom
- Decimate to make smaller
- Resampling to adapt to translations, rotations or non integers scaling

Nearest Neighbor Interpolation: simply take the value the nearest pixel, very bad results

Bilinear Interpolator: weighted coefficients of the 4 nearest pixels (non linear), good compromise between quality and speed

2.2) Geometric and Single Pixel Operations

Geometric Transforms

It works in two steps:

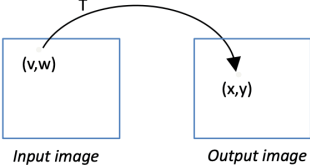
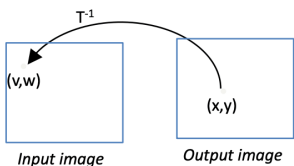
1. Spatial (affine) Transform of Coordinates $(x, y) = T(v, w)$
2. Resampling

An affine transform is a linear matrix operation on the pixel values:

$$\begin{bmatrix} x \\ y \end{bmatrix} = A \begin{bmatrix} v \\ w \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} \\ t_{21} & t_{22} \end{bmatrix} \begin{bmatrix} v \\ w \end{bmatrix} + \begin{bmatrix} t_{13} \\ t_{23} \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = T \begin{bmatrix} v \\ w \\ 1 \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ w \\ 1 \end{bmatrix}$$

a multitude of transformations can be done: scaling, rotation, shearing, affine warping
However the coordinates are fractional and resampling is needed

	
Forward mapping	Backward mapping
$(x, y) = T\{(v, w)\}$	$(v, w) = T^{-1}\{(x, y)\}$
For each (v, w) compute corresponding (x, y)	For each (x, y) compute corresponding (v, w)
Ambiguity issue: multiple points on the same output location (x, y)	Compute each (x, y) one and only one time
(x, y) decimals $\rightarrow$ resampling is needed (e.g., bilinear)	(v, w) decimals $\rightarrow$ resampling is needed (e.g., bilinear)
Missing pixels	Fills all pixels
Less used	Better!

Forwards vs Backwards mapping

Spatial Operations

Intensity Trasformation

This is the act of changing the pixel intensity values with a function
For example the **negative of a grayscale image** is

$$r' = T(r) = (L - 1) - r$$

Gamma Transformation

$$r' = cr^\gamma$$

where $c = (L - 1)^{1-\gamma}$

- $\gamma > 1$: image gets darker
- $\gamma < 1$ brighter

HISTOGRAM

The normalized histogram of an image represents the pixel intensity density of that image

$$p(r_k) = \frac{n_k}{MN}$$

This is a Probability Density Function (PDF)

The usual approach is to have a base histogram image and to force an arbitrary image to change its histogram to match the one of the base image.

The dynamic range can be adjusted via a **histogram tuning** approach:

- First compute the Probability Density Function (PDF) as

$$p_r(r_k) = \frac{n_k}{MN}$$

where n_k is the number of pixels with intensity r_k

- Map the values to the CDF

$$s_k = T(r_k) = (L - 1) \sum_{i=0}^k p_r(r_i) = \frac{L - 1}{MN} \sum_{i=0}^k n_i$$

- Compute the PDF and CDF of the target function so that

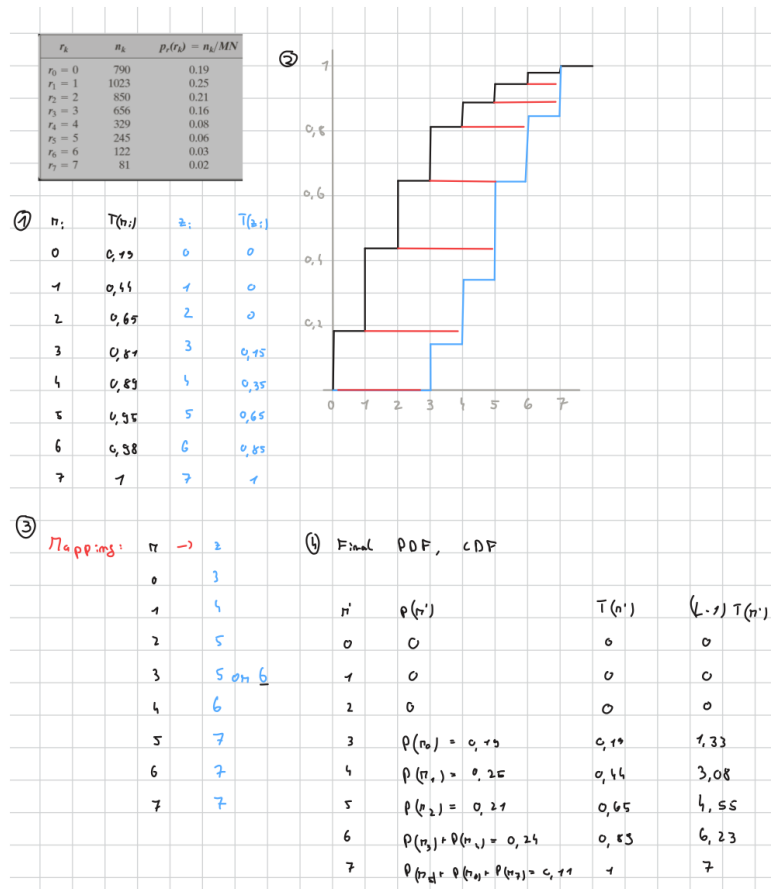
$$G(z_q) = s_k \rightarrow z_q = G^{-1}(s_k)$$

In practice the last step isn't very easy to do so just do the following:

- Compute $T(r)$ and $G(z)$ tables and find pairs $T(r_i) \rightarrow G(z_j)$, that are the ones with closest value.
- Now the final PDF is obtained by plotting the PDF value of $T(r_i)$ in the intensity q of $G(z_q)$. If more than one $T(r_j)$ is assigned to the same $G(z_q)$ then just sum the PDF values and the mapping is obtained by checking what $s_k \rightarrow z_q$, that is $k \rightarrow q$.

We use a target function as a flat histogram isn't necessarily good (ex. night photo must be dark)

Example:



Example

Which condition must a histogram equalization function satisfy for invertibility? Monotonically increasing function

2.3) Processing based on the neighborhood of a pixel

2.3.1) Linear Filters

Spatial Domain

LOW PASS FILTERS

2D convolutions are a form of linear filtering:

A square odd matrix is used as a kernel, where the center element is the pixel and the surrounding values are used to compute the **weighted average of the samples in the window** to which replace the value of the center pixel.

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \frac{1}{16} \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Example

The sum of the weights must be equal to 1.

If this is separable the computation time goes from $O(MN(a+b))$ to $O(MNab)$.

This is a low pass FIR filter, it reduces noise but blurs images and removes small details

HIGH PASS FILTERS

Just the negative of the LPF:

$$H_{hp}(u, v) = 1 - H_{lp}(u, v)$$

Frequency Filtering

LOW PASS FILTERS

The DFT and its inverse (IDFT) treat images as if they are **periodic signals**. When you convolve an image with a filter in the frequency domain, the math assumes the image repeats infinitely in all directions. The circular convolution therefore causes some aliasing called **wraparound error**.

This can be avoided by doing zero padding, so that the circular convolution doesn't mathematically interfere with the image.

- **1D Case:** If the image $f(x)$ has length A and the filter $h(x)$ has length C , the padded length P must be:

$$P \geq A + C - 1$$

- **2D Case:** For an image of size $A \times B$ and a filter of size $C \times D$, the padded dimensions P and Q must be:

$$P \geq A + C - 1 \quad Q \geq B + D - 1$$

However this introduces abrupt discontinuities at the edges of the image that can result in **ringing artifacts**

Feature	Ideal Filter	Butterworth Filter	Gaussian Filter
Mathematical Formula ($H(u, v)$)	1 if $D(u, v) \leq D_0$; 0 if $D(u, v) > D_0$	$\frac{1}{1 + [D(u, v)/D_0]^{2n}}$	$e^{-D^2(u, v)/2D_0^2}$
Parameters	Cut-off frequency D_0	Cut-off frequency D_0 and order n	Cut-off frequency D_0 (standard deviation in frequency domain)
Frequency Selection	Perfect/Sharp frequency selection	Better selection as order n increases	Not very precise frequency selection
Ringling Artifacts	Prominent ringing artifacts	Ringling increases with the filter order n	No ringling artifacts
Spatial Response	Infinite length in the spatial domain	Order-dependent; $n = 20$ is close to ideal	The Fourier Transform of a Gaussian is also a Gaussian
Practical Usage	Rarely used due to severe ringing	Provides a tradeoff between selection and ringing	Very commonly used in practice

Recall that the DFT/IDFT of a gaussian is also a gaussian thus teh filter also works in the spatial domain, however notice that the relation of the $D_0 = \sigma$ is inversly proportional in space or frequency

- σ in frequency small = in time big: lot of blurring
- σ in frequency big = in time small: less blurring

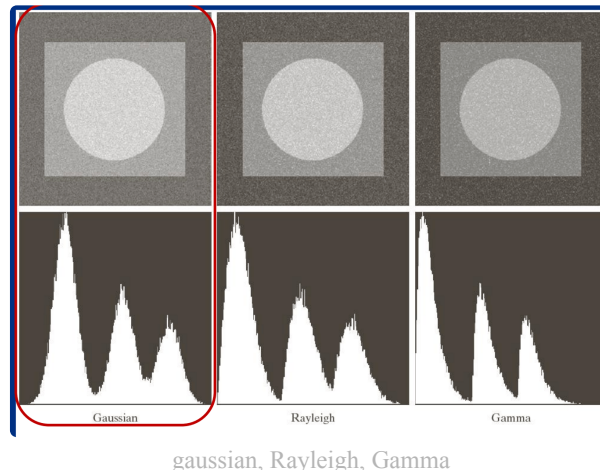
HIGH PASS FILTERS

Same as LPF but with different results

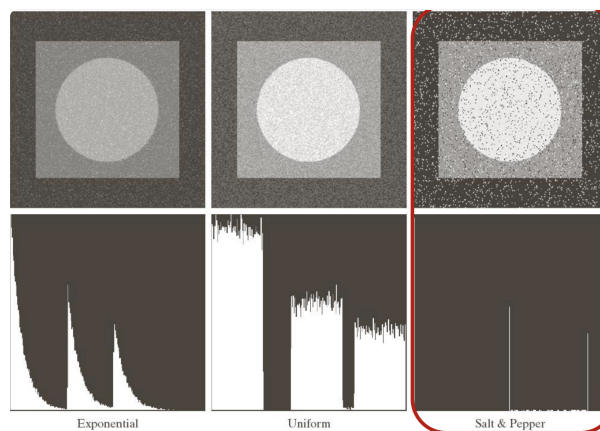
Feature	Ideal High-Pass Filter	Butterworth High-Pass Filter	Gaussian High-Pass Filter
Mathematical Formula ($H(u, v)$)	0 if $D(u, v) \leq D_0$; 1 if $D(u, v) > D_0$ +2	$\frac{1}{1 + [D_0/D(u, v)]^{2n}}$ +1	$1 - e^{-D^2(u, v)/2D_0^2}$ +1
Visual Effect	Sets uniform regions to 0, keeping only edges and details	Results are much smoother than those obtained with an Ideal filter	Provides smooth sharpening without artifacts
Sharpness	Very sharp transition leading to harsh edges	Sharpness is controlled by the order n	Gradual transition; least "sharp" but most natural-looking

2.3.2) Non Linear Filters

Noise models:



gaussian, Rayleigh, Gamma



Exponential, Uniform, Salt & Pepper

MEDIAN FILTER

It is a LPF that assigns the median value. It removes salt and pepper noise, however destroys small structures

MAX AND MIN FILTERS

Max highlights salt and removes pepper noise

Min highlights pepper and removes salt noise

Does not preserve avg intensity

BILATERAL FILTER

LPF that preserves edges

it "enhances" gaussian filtering: the normalized gaussian function is

$$G_{\sigma}(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp \left\{ -\frac{x^2}{2\sigma^2} \right\}$$

gaussian uses space, now we add range

$$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in S} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|) G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|) I_{\mathbf{q}}$$

where

- $W_{\mathbf{p}}$ is a normalization factor for the sum =1
- G_{σ_s} is the standard space gaussian with $\|\mathbf{p} - \mathbf{q}\|$ the euclidean distance between two pixels, everything smaller than σ_s is averaged out
- G_{σ_r} is the abs of the difference between the intensity levels of the two pixels if $\sigma_r \rightarrow \infty$ then the filter is a gaussian filter, otherwise σ_r is the minimum edge amplitude to preserve the edge

3) Middle level processing

Quick recap on discrete derivatives:

First order derivative:

$$f_x = \frac{df}{dx} = f(x+1) - f(x) \quad \text{or} \quad f_x = \frac{df}{dx} = \frac{f(x+1) - f(x-1)}{2}$$

Operator	Formula	Linear
Gradient	$\nabla = [f_x, f_y]$	Yes
Gradient Module	$\sqrt{f_x^2 + f_y^2}$	No
Approx Module	$\ f_x\ + \ f_y\ $	No

Second order:

$$f_{xx} = \frac{d^2 f}{dx^2} = f(x+1) + f(x-1) - 2f(x)$$

Second order derivatives are used to compute the Laplacian filter:

$$\nabla^2 f = f_{xx} + f_{yy}$$

this is used to compute edges via zero crossing and is isotropic (response the same in all directions)

3.1) Edge Detection

Basic Edge Detectors

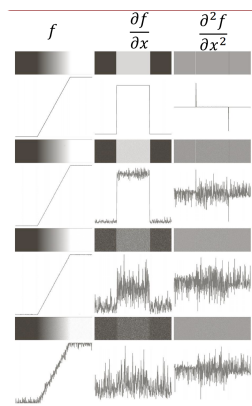
Edges are caused by many phenomena:

- Mathematically they are caused by: steps, ramps, roofs
- In CV they are present in: Surface discontinuity, depth discontinuity, light or color discontinuity

How do derivatives compare at detecting edges?

	1st Order	2nd Order
Large Values	All the edge	Start and end of edge
Response Type	Single	Double
Response Thickness	Thick	Thin
Noise Sensitivity	Moderate	High

Notice that derivatives (especially 2nd order) are sensitive to noise, therefore the gradient is used



Example: Noise Sensitivity

The gradient points in the direction of most rapid change in intensity and thus it is perpendicular to the edge

Basic Edge Detection idea: Compute magnitude of gradient and if it is greater than a threshold an edge is detected:

$$M = |\nabla f| > T$$

BASIC MASK

The most basic mask is the following

$$\frac{\partial f}{\partial x} \begin{bmatrix} 1 & -1 \end{bmatrix} \quad \frac{\partial f}{\partial y} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Mask

Very fast and simple but with samples half way between the pixels and not robust to noise

SOBEL MASK

A more advanced technique is the Sobel mask

$$\frac{\partial f}{\partial x} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \frac{\partial f}{\partial y} \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Mask

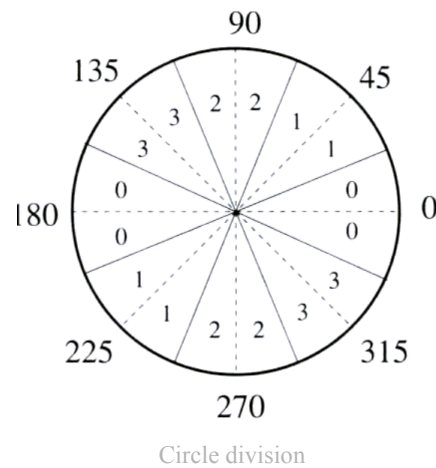
It is centered on the pixel, more robust to noise but not too precise

A LPF can help improve the quality of the edge detector. edge detectors are high pas filters, however edges are of a lower frequency than noise and thus a LPF can remove noise without damaging the edges excessively

Canny Edge Detector

The idea is to check if the edge (pixel) is a local maxima in the direction of the gradient (edge). Instead of interpolating between all possible locations canny uses a trick

- Assign angle of gradient $\alpha(x,y)$ to one of the 4 sectors (divide circle in $4 \cdot 2$ areas)
- Check 3x3 area around the pixel
- If the value at the center is smaller than at least one of the 2 values along the quantized gradient direction, then $M(x,y)$ is set to 0



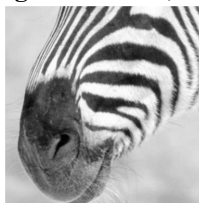
- **Hysteresis threshold:** Big gradient values are an edge for sure, small gradient values are an edge only if the for a path between bigger values otherwise they are discarded

Standard canny implementation uses a preliminar low pass gaussian filter (explained before why)

- **Small kernel size:** more granular details are present, however fake noisy edges might be present.
- **Small kernel size:** removes granular detail and smooths out edges.

Hysteresis threshold can have 3 possible cases:

- **Both Low:** all borders are present, even smaller ones, lot of noise!
- **Both High:** Keep only strong gradients
- **T_l low, T_h high:** Ideal case, keep strongest borders but also keep smaller ones close to strong borders



1 T_l low
 T_h low



2 T_l high
 T_h high



3 T_l low
 T_h high

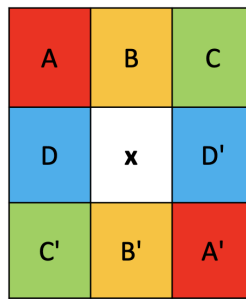
Example

Marr-Hildreth Edge Detector

Exploits second derivative and therefore is sensitive to noise but avoids double edges because of the **zero crossing** constraint

It uses the Laplacian of a Gaussian (LoG), but an additional zero crossing and threshold check is required,

- As usual divide 3x3 area around the pixel
- Zero crossing: there is an edge if $XX' < 0$ (opposite sides)
- Threshold: $|X - X'| > T$



Grid

The locuses of the zero crossing have a closed loop shape

3.2) Corner Detection

A corner differs from an edge since the content varies in all directions. Moreover, edges are localized in one direction while coreners are localized in two directions, a specific (x, y) point.

Target properties for a corner detector:

- Accurate localization
- Invariance wrt shift, rotation, scale, brightness change
- High repeatability

Harris Corner Detector

Works in 3 steps:

1. Analyze gradient to detect response to any shift
2. Eigenvalue decomposition to get the orientation of change and distinguish a corner from an edge
3. Detect the corner by comparing the eigenvalues with a threshold

Step 1:

Define sensitivity within a window at point (x_i, y_i)

$$s(\Delta x, \Delta y) = \sum_{x,y \in W} w(x - x_i, y - y_i) [i(x + \Delta x, y + \Delta y) - i(x, y)]^2$$

With the 1st order taylor approx we have

$$i(x + \Delta x, y + \Delta y) \approx i(x, y) + \frac{di}{dx} \Delta x + \frac{di}{dy} \Delta y$$

In matrix form it becomes

$$\begin{aligned} s(\Delta x, \Delta y) &= \sum_W g() \cdot \left[\begin{bmatrix} \frac{di}{dx} & \frac{di}{dy} \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \right]^2 \\ s(\Delta x, \Delta y) &= [\Delta x \ \Delta y] \begin{bmatrix} \sum g \frac{di}{dx}^2 & \sum g \frac{di}{dx} \frac{di}{dy} \\ \sum g \frac{di}{dx} \frac{di}{dy} & \sum g \frac{di}{dy}^2 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = [\Delta x \ \Delta y] \begin{bmatrix} A & C \\ C & B \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \\ &= [\Delta x \ \Delta y] M \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \end{aligned}$$

Step 2:

Let λ_1, λ_2 be the eigenvalues of M and v_1, v_2 the eigenvectors, then the eigenvectors measure the strength of the response to teh two perpendicular directions

Step 3

- **Both large:** Corner

- **One much bigger than the other:** Edge
- **Both small:** Interior

A faster approximation is obtained by computing

$$R(x, y) = \det(M) - k \cdot \text{tr}(M)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

- $R \gg 0$ Corner
- $R < 0$ Edge
- $R \approx 0$ Interior

Robustness analysis:

- Invariant to **constant** brightness change
- Invariant to shift and rotation
- NOT invariant to complex illumination
- NOT invariant to scaling

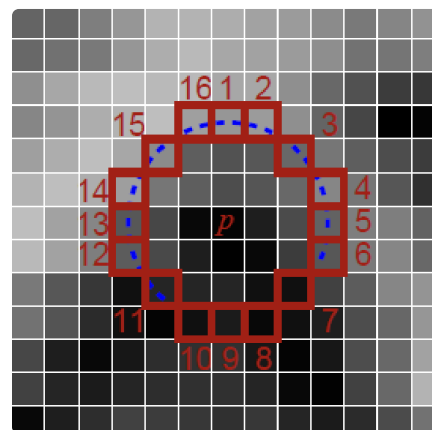
Features from Accelerated Segment Test (FAST)

This is a very fast approach:

- Look at circle of N pixels around pixel
- There is a corner if there is a continuous arc S of n contiguous pixels all much brighter/darker than p
- The intensity of all pixels must be larger/lower than the pixel intensity $\pm$ a threshold, that is

$$\forall x \in S : I_x > I_p + T \text{ or } \forall x \in S : I_x < I_p - T$$

usually $N = 16, n = 12 \rightarrow 75\%$ of circle



Example

By checking only on 1 and 9 we can rapidly reject the pixel if none of the two satisfy the condition

Same for 1,9 and 5 where two must satisfy the condition

Same for 1,9,5 and 13 where three must satisfy the condition

Some double points will be present, these can be removed via non maxima suppression

3.3) Hough Transform (Line Recognition)

A brute force approach that would require to compute edges and then find all lines passing through all points and see if these are compatible $O(n^3)$

Consider the equation of a line and invert its variables and parameters and this representation is the **parameter space**:

$$y(x) = ax + b \rightarrow b(a) = -xa + y$$

this means that given a point, we get all the lines passing through it.

If two lines are not parallel, they meet in a unique point in the parameter space

Normalize the representation so to allow the vertical lines $m \rightarrow \infty$, the representation is

$$y = -\frac{\cos \theta}{\sin \theta} x + \frac{\rho}{\sin \theta} \rightarrow \rho = x \cos \theta + y \sin \theta$$

And thus a point in the image is represented by a sinusoidal

Therefore the working is

- A point in the image has infinite lines passing through it
- 2 points have only one line passing through them
In parameter space this is:
- A line in parameter space has infinite points
- Two lines intersect at a single point
- The hough line detector computes the parameter representation for each pixel (x, y)
- Via a clustering algorithm it recognizes the strongest intersections and returns (ρ, θ)
- These are the parameters of the lines

The parameter space is quantized

- θ is finite and divided so to have similar angles together
- ρ is infinite and therefore it is clamped and quantized, limits the max diagonal in the image

This allows to handle point not perfectly aligned.

Once the edge points are found, the cells that contain them in the parameters space get counted up by 1, the cell with most counts is the line

3.4) Feature Detection

Lets start with the definition of feature detector

Definition 7 (Feature Detector).

A feature detector (extractor) is an algorithm taking an image as input and outputting a set of regions ("local features")

Definition 8 (Local Features).

Local features are regions (arbitrary set of pixels) that are

- **Distinguishable** regardless of viewpoint, illumination, scale, transforms etc
- Robust to occlusion
- Discriminative neighbourhood

Definition 9 (Feature Descriptor).

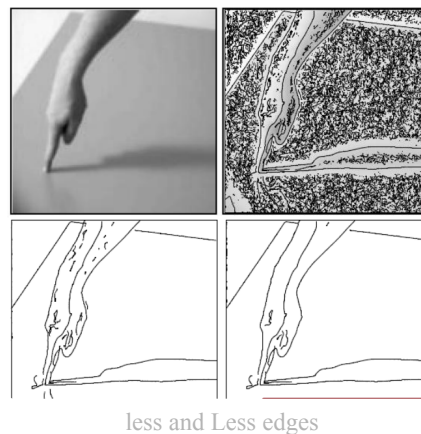
A descriptor is computed on an image region defined by a detector and provides a numerical representation of the intensity or color function within that region, typically expressed as a vector of values.

SCALE SPACE (NOT IN EXAM)

Invariance to scale is harder to achieve as different scales highlight different features

Scale space is a set of signals parameterized by a continuous scale parameter $f(x, y) \rightarrow f(x, y, t)$ where smaller structures are progressively suppressed when the scale increases

The gaussian scale space is defined by the kernel size of the gaussian filter applied to the image. This blurs the image more and more, a scale can be found for example by looking at what gives the X strongest edges.



To select the correct scale for feature extraction, image derivatives must be normalized because their magnitude increases at smaller scales. Each derivative operation is normalized by multiplying by $t^{\{\gamma/2\}}$, applied once per derivative order. Features are detected at all scales, and the strongest responses of the normalized derivatives are selected as the final features.

Scale Invariant Feature Transform (SIFT)

Milestone in computer vision, it works in 4 steps

1. Scale-Space extrema detection
2. Keypoint localization
3. Orientation assignment
4. Keypoint descriptor

Step 1: Scale-Space extrema detection

First divide into octaves:

- Octave 1 is original image and set $\sigma = 1$
- Octave 2 is resized image (half resolution) and set $\sigma = 2$
- Octave n is $n - 1$ times resized (resolution / 2^{n-1}) image and set $\sigma = 2^{n-1}$

Inside each octave the following is done:

- Compute Gaussian Blur of $s + 3$ images starting from the octave image with $\sigma_s = 2^{1/s} \sigma_n$
- Compute DoG to approximate LoG, that is: Subtract this level with upper level

These $s + 3$ form a stacked grid and

- at each DOG output level (except for first and last) the minima and maxima of the DoG is located.
- This point is compared with the 8 neighbors and the 9 neighbors on upper and lower scales
- If it is the biggest/smallest of all these points it is a keypoint

Step 2: Keypoint localization

Execute a Second order Taylor series approximation of the DoG in the neighbourhood of the point to accurately locate the maxima, the zero crossing point is the accurate location. Finally discard points with $|D(x, y, \sigma)| < \text{threshold}$

Also remove edge points using the hessian matrix (similar as harris corner detection)

1. $H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$ (Hessian Matrix)
2. $Tr(H) = D_{xx} + D_{yy} = \lambda_1 + \lambda_2$
3. $Det(H) = D_{xx}D_{yy} - (D_{xy})^2 = \lambda_1\lambda_2$
4. Define $r = \frac{\lambda_1}{\lambda_2} \rightarrow \lambda_1 = r\lambda_2$
5. $\frac{Tr(H)^2}{Det(H)} = \frac{(\lambda_1 + \lambda_2)^2}{\lambda_1\lambda_2} = \frac{(r\lambda_2 + \lambda_2)^2}{r\lambda_2^2} = \frac{(r+1)^2}{r}$
6. Condition for corner: $\frac{Tr(H)^2}{Det(H)} = \frac{(r+1)^2}{r} < T$

Corner detection

but keep corner points if possible

Step 3: Orientation assignment

To compute the orientation of the key point

- the gradient in the neighborhood is computed, then these are quantized into 36 bins of 10 degrees each
- A histogram is computed with weight based on the distance from kp
- The orientation is the peak value in the histogram
 - If the two highest peaks are too close ($p_2/p_1 > 0.8$) then multiple kp are created with different orientations
- A parabola can be fitted on the 3 peaks for more accurate orientation
- 4 stable coordinates for each kp (x,y,scale,orientation)

All the subsequent operations for the KP will be made on an image:

- Rotated with respect to the found orientation
- Scaled according to the keypoint scale

This makes the description invariant with respect to scale and rotation

Step 4: Keypoint descriptor

Around each keypoint, gradient magnitude and orientation are computed in a 16×16 neighborhood and weighted by a Gaussian. The region is split into 4×4 blocks, each producing an 8-bin orientation histogram, and all histograms are concatenated into a 128-dimensional descriptor. Trilinear interpolation is applied across the three dimensions (x, y, orientation) for stability. The descriptor is then normalized to 1 for illumination robustness, thresholded at 0.2 to limit large components caused by artifacts, and finally renormalized to 1.

To match SIFT features of an object and SIFT of the image there are two approaches

- Simple approach: SIFT of object, SIFT of image, then match with minimum distance. This results in too many false matches
- Apply distance threshold. Moreover, since similar SIFT features are linked together, if two keypoints are matched, then discard the match if their ratio is > 0.8

If an image has had an affine transformation applied, it is possible to estimate the transformation by matching the same features in the before and after images and use the coordinates with least squares approximation

Local Binary Patterns (LBP)

Used for texture recognition

Compare the central pixel in a 3x3 window with the 8 samples around it and build a binary sequence based on the signs of the differences with center pixel

This sequence will be a number, then build histogram of these numbers

Binary Robust Independent Elementary Features (BRIEF)

Apply gaussian smoothing and then compare pixels inside a window p and build a binary string with the comparison output. XOR is used for comparison of descriptors

Oriented FAST and Rotated BRIEF (ORB)

Uses FAST corner detector and BRIEF descriptor

- Build pyramid of images at different resolutions
- Detect corner points
- using Harris criteria detect best features
- Assign orientation wrt intensity centroid and central pixel
- compute BRIEF on the image rotated according to computed orientation

Method	Detector	Descriptor	Translation Inv.	Rotation Innv.	Scale Inv	Computation Time
SIFT	Yes	Yes	Yes	Yes	Yes	slow
LBP	No	Yes	Yes	No	No	fast
BRIEF	No	Yes	Yes	No	No	fast
ORB	Yes	Yes	Yes	Yes	Yes	fast

3.5) Image Segmentation

Start with a definition:

Definition 10 (Segmentation).

Subdivide an image into a set of regions corresponding to the different objects or regions in it, formally:
Subdivide an image into R regions and Q a logical predicate such that:

- $\bigcup_i R_i = R$ (each pixel assigned to one region)
- $R_i \cap R_j = \emptyset$
- (optional) R_i is a connected set
- $Q(R_i) = \text{true} \forall i$
- $Q(R_i \cup R_j) = \text{false}$ (predicate holds exclusively in one region)

It is based around similarity of the pixels in same area, and discontinuity between regions. However **no true solution exists**

Histogram Approach

Build Histogram of image and find two regions based on threshold in histogram

Noise destroys histogram

Otsu's Method:

The optimal threshold is the one that maximizes inter-class variance, that is:

- Compute histogram
- Compute the inter class variance:

$$\sigma_b^2 = P_1(k)(m_g - m_1(k))^2 + P_s(m_g - m_2(k))^2$$

with P the probability of the pixel belonging to that class and m the mean intensity of the image and in the classes

- find k that maximizes the variance
- segment image

Clustering Based Approach

Represent each pixel by a vector of numbers and cluster them

K-MEANS CLUSTERING

recall [Intro to ML > 2\) Unsupervised Learning \(Clustering\)](#). Here is a quick recap:

Inputs:

Data points $x_1, \dots, x_m \in \mathbb{R}^d$ and $k \in \mathbb{N}^+$

Goal:

find

- Partitions $C = (C_1, \dots, C_k)$ of $x_1, \dots, x_m$
- Centers $\mu_1, \dots, \mu_k \in \mathbb{R}^d$ that minimize cost

$$\min_{\mu_j \in \mathbb{X}' \forall j} \sum_{i=1}^k \sum_{x \in C_i} d(x, \mu_i)^2$$

A common algorithm is very simple:

Theorem 11 (Centroid).

Given a cluster C_i , we call **centroid** the center μ_i that minimizes $\sum_{x \in C_i} d(x, \mu_i)^2$, that is:

$$\mu_i(C_i) = \arg \min_{\mu \in \mathbb{X}'} \sum_{x \in C_i} d(x, \mu)^2 = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

1. Find random centroids and assign closest points to them
2. Recalculate centroid position and reassign points
3. Repeat step 2 until convergence since at each iteration the cost function does not increase

This algorithm is fast and simple and always converges but it doesn't yield optimal results and the amount of clusters must be known a priori

MEAN SHIFT CLUSTERING (TODO)

The mean shift algorithm seeks modes or local maxima of samples' density in the feature space

Intuitively this resolves to finding the densest region, to do so

- generate random center with ROI of radius r .
- Calculate center of mass and move center to that position
- Repeat until convergence

This can be used for a non-parametric density estimation since by assumption the data points are sampled from a PDF. If parametric the PDF is a sum of kernel functions (scaled + shifted multidimensional gaussian)

This can also be used for a non parametric density gradient estimation. **The gradient is proportional to a scaled version of the Mean Shift vector**

4) High level algorithms

Object detection works by localizing and classifying objects in an image.

ML is used. First the image has feature extracted, these features get classified by a ML and the result is a label

4.1) Bag of Words Models

Textures are made of repetitions of basic elements (textons). Classifying a texture is equivalent to identifying its texton. Analogously for bag of words. An image is a collection of objects and classifying the images is equivalent to classifying the objects.

Features are extracted and labeled to a visual vocabulary

1. Key-Patch Detection (feature detection)
2. Feature extraction and build bag of features and cluster it (also histogram)
3. The visual vocabulary is the set of clusters, that will result in a bag of normalized patches

4.2) Viola And Jones Object Detection (Face Detection)

The idea is simple, slide a window through the image and evaluate face models at every location at every scale. This requires tens of thousand of evaluations and we aim at a false positive rate of 10^{-6} .

It works in 3 steps:

1. Integral images (feature evaluation)
2. Boosting (feature selection)
3. Cascade of classifier (fast reject)

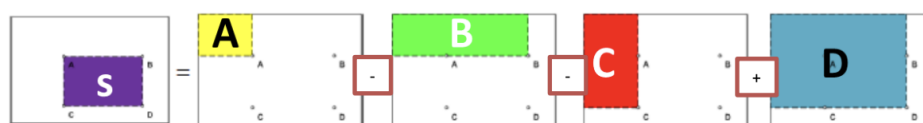
Step 1: Integral images

The type of feature we extract are called **Haar features**. These features calculate the difference between the sum of pixels in adjacent black and white rectangular regions.

$$f(x) = \sum_i p_b(i) - \sum_i p_w(i)$$

from 2-rectangle to 4-rectangle features.

In a 24x24 image there are 160000 combinations and we need to evaluate them quickly. Therefore we use **integral images** which allow to evaluate the Haar feature with only 4 array operations for every pixel.



Integral Images

It can be computed in one pass over the image.

Very simple features, but very fast and can generate many so to also scan at many scales.

The scale is determined by the rectangle feature size, not the image actually being rescaled.

Step 2: Ada Boost

Define a weak classifier of a haar feature as a function that returns ± 1 .

$$h_t(x) = \begin{cases} +1 & \text{if } p_t f(x) > p_t \theta_t \\ -1 & \text{otherwise} \end{cases}$$

Then the classifier is a weighted sum of the weak learners. The weights depend on the accuracy

$$h(x) = \text{sign} \left[\sum_j a_j h_j(x) \right]$$

Boosting is part of the training phase.

- Input: training set of N labeled images
- 1. Start with equal weights for all classifiers. ($w_{i,j}$ =i-th weight in j-th iteration)
 1. Select the best weak classifier, the one that minimizes weighted classification error, that is the minimum weighted error on all images

$$e_j = \sum_{i=0}^{N-1} w_{i,j} e_{i,j}$$

2. update the weights $w_{i,j+1} = w_{i,j} \left(\frac{e_j}{1-e_j} \right)^{1-e_{i,j}}$ and renormalize them
3. Compute the weight of the weak classifier

$$a_j = -\log \left(\frac{e_j}{1-e_j} \right)$$

2. Repeat these steps until convergence

Step 3: Cascade of classifiers

Instead of implementing a monolithic classifier, we implement a cascade of classifiers that perform similarly on accuracy but are 10 times faster.

Each classifier is trained with more features and becomes more complex, however the more complex classifiers have less samples to pass since they were rejected by the previous classifiers.

Start from 2 features (reject 50% of outliers), in total 32 steps and 4297 features

4.3) Intro To Deep Learning

Prior to deep learning, feature extraction was found by an algorithm and the classification was done by a neural network. Deep learning merges feature extraction and classification into a single neural network.

The neural network is a feed forward network organized into layers and is a directed graph with no cycles.

Each neuron takes in input the sum of the outputs of the connected neurons from previous layer weighted by the edge weights (plus a bias) and applies a simple scalar function σ

$$a_i(l) = \sigma \left(\sum_j w_{ij}(l) a_j(l-1) + b_i \right)$$

usually features are smaller than the image and can be present in various different locations, so we introduce the CNN.

1. Local connectivity
Each neuron is connected to a subset of neurons from the previous layer
2. Shared weights
All neurons share the same weights to the previous layer (convolution with 1D filter) (spatial invariation)
3. Multiple feature maps
Instead of a succession of layers, some layers can be double, that is same convolution with different input windows (weights)
4. Pooling layers
Reduce resolution so that the next convolutional layer is applied at a larger scale

It can learn hierarchical representation of data (at multiple scales).

4.4) Deep Learning for Image Classification

Input: Image

Output: label for entire image (subject of image)

CNN using CIFAR dataset has 78% accuracy, however the error are very close to what it can look like to a human

Modello	Anno	Caratteristiche Principali	Error Rate (ImageNet)
AlexNet	2012	Prima ad utilizzare ReLU, Dropout e Data Augmentation; 5 layer conv e 3 FC.	15.3%
GoogLeNet	2014	Introduce i moduli Inception (convoluzioni parallele); 22 layer ma meno parametri di AlexNet.	6.67%
VGGNet	2014	Architettura molto uniforme con filtri piccoli (3×3) ma numerosi (138M di parametri).	-
ResNet	2015	Introduce le skip connections (connessioni identità) per addestrare reti profondissime (fino a 152 layer).	3.57%
DenseNet	2017	Ogni livello riceve input da tutti i livelli precedenti tramite concatenazione.	-

4.5) Deep Learning for Object Detection

Given an image, the goal is to detect the object and locate its position in the image

dataset has already positioned boxes with labels

- **True Positive (TP):** A correct prediction where the predicted box matches the Ground Truth. While not explicitly labeled as "50%" in all text, the slide shows Intersection Over Union (IoU) < 0.5 leads to failure.
- **False Positive (FP):** This occurs when the detector predicts an object that isn't there, uses the **wrong class** label for an object, or has a box with **insufficient overlap** with the ground truth.
- **False Negative (FN):** This occurs when there is **no prediction** for an actual object in the scene or the overlap is so poor it is considered a miss.

R-CNN, how does it work:

- **Region Proposals:** The model creates ~2000 bounding boxes (region proposals) using **Selective Search**, which groups pixels by texture, color, and intensity.
- **Warping:** Each proposal is scaled (warped) to a standard square size.
- **Feature Extraction:** The warped regions are passed through a **CNN** (a modified AlexNet) to compute features.
- **Classification & Regression:** A **Support Vector Machine (SVM)** classifies the object within the region.
 - A **Bounding-box regressor (Bbox reg)** is used to refine the coordinates of the box.

Model	Key Innovation	Speed/Efficiency
R-CNN	Selective Search + CNN + SVM	Slow: Must pass ~2000 proposals through the CNN individually.
Fast R-CNN	RoIPool (Region of Interest Pooling)	Faster: Only one CNN pass for the whole image; shares features across regions.
Faster R-CNN	Replaces Selective Search with a Neural Network .	Much Faster: Region proposals are learned.
YOLO	Treats detection as a single regression problem .	Real-time: Runs the CNN only once; predicts all boxes and classes simultaneously.

Model	Key Innovation	Speed/Efficiency
SSD	Single-shot detector using multi-scale feature maps.	Efficient: Predicts boxes and classes in a single pass.

4.6) Deep Learning for Image Segmentation

Semantic segmentation assigns one label to each pixel and the same labels belong to same region

Fully Convolutional NN (FCN) are used. They remove the final fully connected layer (this imposes fixed resolution) and connect i-th layer with all previous layers. Skipped connections are used to connect layers with different resolutions

4.7) DL Recap

Technique	Function	How it Works
Convolution	Feature Extraction	Slides a learnable filter (kernel) over the image to create feature maps that highlight specific patterns.
ReLU (Rectified Linear Unit)	Non-Linearity	Applies an element-wise activation function (typically $f(x) = \max(0, x)$) to introduce non-linearity, allowing the network to learn complex data.
Pooling (Max/Avg)	Downsampling	Reduces spatial dimensions (rescaling) by selecting the maximum or average value in a local window, providing translation invariance.
Fully Connected (FC)	Classification	Connects every neuron in one layer to every neuron in the next; typically used at the end of the network to map extracted features to class scores.
Dropout	Regularization	Randomly "drops" (sets to zero) a percentage of neurons during training to prevent the model from over-relying on specific paths (overfitting).
Softmax	Probability Output	Converts the raw output scores (logits) of the network into a probability distribution over the predicted classes.
Skip Connections (Residuals)	Gradient Flow	Adds the input of a layer to its output ($F(x) + x$), allowing gradients to flow through the network more easily and enabling the training of very deep networks.